St. Francis Institute of Technology, Mumbai-400 103
Department Of Information Technology

A.Y. 2025-2026
Class: BE-IT A/B, Semester: VII

Subject: Secure Application Development Lab

Student Name: Tanmay Bhatkar Student Roll No: 14

Experiment – 10 : Study and Implement Hashing Algorithm

Aim: To study and implement hashing algorithm
Objectives: After study of this experiment, the student will be able to

- understand role of hashing function in secure coding
- understand the concept of hashing function
- understand practical applications of hashing algorithm

Lab objective mapped: ITL703.6 To **apply** secure coding for cryptography

Prerequisite: Basic knowledge of hashing function in cryptography.

Requirements: C/C++/JAVA/PYTHON

Pre-Experiment Theory:

Hash Function is a function that has a huge role in making a System Secure as it converts normal data given to it as an irregular value of fixed length. When we put data into this function it outputs an irregular value. The Irregular value it outputs is known as **"Hash Value"**.
Hash Values are simply numbers but are often written in Hexadecimal. Computers manage values as Binary. The hash value is also data and is often managed in Binary. A hash function is basically performing some calculations in the computer. Data values that are its output are of fixed length. Length always varies according to the hash function. Value doesn't vary even if there is a large or small value.
If given the same input, two hash functions will invariably produce the same output. Even if input data entered differs by a single bit, huge change in their output values. Even if input data entered differs huge, there is a minimal chance that the hash values produced will be identical. If they are equal, it is known as **"Hash Collision".** Converting Hash Codes to their original value is an impossible task to perform. This is the main difference between Encryption as a Hash Function.

Features of hash functions in system security:

One-way function: Hash functions are designed to be one-way functions, meaning that it is easy to compute the hash value for a given input, but difficult to compute the input for a given hash value. This property makes hash functions useful for verifying the integrity of data, as any changes to the data will result in a different hash value.

Deterministic: Hash functions are deterministic, meaning that given the same input, the output will always be the same. This makes hash functions useful for verifying the authenticity of data, as any changes to the data will result in a different hash value.
Fixed-size output: Hash functions produce a fixed-size output, regardless of the size of the input. This property makes hash functions useful for storing and transmitting data, as the hash value can be stored or transmitted more efficiently than the original data.
Collision resistance: Hash functions should be designed to be collision resistant, meaning that it is difficult to find two different inputs that produce the same hash value. This property ensures that attackers cannot create a false message that has the same hash value as a legitimate message.
Non-reversible: Hash functions are non-reversible, meaning that it is difficult or impossible to reverse the process of generating a hash value to recover the original input. This property makes hash functions useful for storing passwords or other sensitive information, as the original input cannot be recovered from the hash value.

Advantages:
Data integrity: Hash functions are useful for ensuring the integrity of data, as any changes to the data will result in a different hash value. This property makes hash functions a valuable tool for detecting data tampering or corruption.
Message authentication: Hash functions are useful for verifying the authenticity of messages, as any changes to the message will result in a different hash value. This property makes hash functions a valuable tool for verifying the source of a message and detecting message tampering.
Password storage: Hash functions are useful for storing passwords in a secure manner. Hashing the password ensures that the original password cannot be recovered from the hash value, making it more difficult for attackers to access user accounts.
Fast computation: Hash functions are designed to be fast to compute, making them useful for a variety of applications where efficiency is important.

Disadvantages:
Collision attacks: Hash functions are vulnerable to collision attacks, where an attacker tries to find two different inputs that produce the same hash value. This can compromise the security of hash-based protocols, such as digital signatures or message authentication codes.
Rainbow table attacks: Hash functions are vulnerable to rainbow table attacks, where an attacker precomputes a table of hash values and their corresponding inputs, making it easier to crack password hashes.
Hash function weaknesses: Some hash functions have known weaknesses, such as the MD5 hash function, which is vulnerable to collision attacks. It is important to choose a hash function that is secure for the intended application.
Limited input size: Hash functions produce a fixed-size output, regardless of the size of the input. This can lead to collisions if the input size is larger than the hash function output size.

Procedure:

1) SHA-1 Hashing Algorithm

```
#SHA-1
def left_rotate(n, b):
    """Left rotate a 32-bit integer n by b bits."""
    return ((n << b) | (n >> (32 - b))) & 0xffffffff

def sha1(data):
    # String to Byte conversion
```

```
    if isinstance(data, str):
        data = data.encode()

    # Padding the message
    original_byte_len = len(data)
    original_bit_len = original_byte_len * 8

    # append the bit '1' to the message to separate the pad and message
    data += b'\x80'

     # append 0 <= k < 512 bits '0', so that the resulting message length (in bits) is congruent to 448
(mod 512)
    while (len(data) * 8) % 512 != 448:
        data += b'\x00'

    # append length of message (before pre-processing), in bits, as 64-bit big-endian integer
    data += original_bit_len.to_bytes(8, byteorder='big')

    # Initialize hash values (h0 to h4)
    h0 = 0x67452301
    h1 = 0xEFCDAB89
    h2 = 0x98BADCFE
    h3 = 0x10325476
    h4 = 0xC3D2E1F0

    # Process the message in successive 512-bit chunks:
    for chunk_start in range(0, len(data), 64):
        chunk = data[chunk_start:chunk_start + 64]

        # Break chunk into sixteen 32-bit big-endian words w[0..15]
        w = [int.from_bytes(chunk[i:i+4], 'big') for i in range(0, 64, 4)]

        # Extend the sixteen 32-bit words into eighty 32-bit words:
        for i in range(16, 80):
            val = w[i-3] ^ w[i-8] ^ w[i-14] ^ w[i-16]
            w.append(left_rotate(val, 1))

        # Initialize hash value for this chunk:
        a, b, c, d, e = h0, h1, h2, h3, h4

        # Main loop:
        for i in range(80):
            if 0 <= i <= 19:
                f = (b & c) | ((~b) & d)
                k = 0x5A827999
            elif 20 <= i <= 39:
                f = b ^ c ^ d
                k = 0x6ED9EBA1
            elif 40 <= i <= 59:
                f = (b & c) | (b & d) | (c & d)
                k = 0x8F1BBCDC
            else:  # 60 <= i <= 79
                f = b ^ c ^ d
                k = 0xCA62C1D6
```

```
            temp = (left_rotate(a, 5) + f + e + k + w[i]) & 0xffffffff
            e = d
            d = c
            c = left_rotate(b, 30)
            b = a
            a = temp

        # Add this chunk's hash to result so far:
        h0 = (h0 + a) & 0xffffffff
        h1 = (h1 + b) & 0xffffffff
        h2 = (h2 + c) & 0xffffffff
        h3 = (h3 + d) & 0xffffffff
        h4 = (h4 + e) & 0xffffffff

    # Produce the final hash value (big-endian) as a hex string
    return '{:08x}{:08x}{:08x}{:08x}{:08x}'.format(h0, h1, h2, h3, h4)

# Example usage:
msg=input("Enter your message to be hashed: ")
print("SHA-1 output: ", sha1(msg))
```

Output:

```
Enter your message to be hashed: Running up the hill with the troops
SHA-1 output:  1b953372368693ac36e17955820148378bc40ea9
```

2) CRC32 (Cyclic Redundancy Check)

Code:

```
# Precompute CRC32 table globally
CRC32_TABLE = []
for i in range(256):
    crc = i
    for _ in range(8):
        if crc & 1:
            crc = (crc >> 1) ^ 0xEDB88320
        else:
            crc >>= 1
    CRC32_TABLE.append(crc)


def crc32(data):
    """
    Compute CRC32 checksum for the input data.
    Arguments:
        data (bytes): Input data as bytes.
    Returns:
        int: CRC32 checksum as an unsigned 32-bit integer.
    """
    crc = 0xFFFFFFFF
    for byte in data:
        crc = CRC32_TABLE[(crc ^ byte) & 0xFF] ^ (crc >> 8)

    return crc ^ 0xFFFFFFFF
```

```
if __name__ == "__main__":
    input_data = input("Enter you message to be hashed: ")
    sample_data = input_data.encode('utf-8')
    checksum = crc32(sample_data)
    print(f"CRC32 Checksum: {checksum:#010x}")  # Print as hex, zero-padded
```

```
= RESTART: C:/Users/Student/Desktop/TB_exp10b.py
Enter you message to be hashed: tanmay bhatkar
CRC32 Checksum: 0x7706d96b
```

Post-Experiments Exercise:

Extended Theory: Nil

Post Experimental Exercise:

Questions:

- Explain the difference between hashing and encryption. In what scenarios would you use one over the other, and why?
- Provide examples of practical applications of hashing functions in computer science beyond data integrity and password storage.
- Briefly explain how hashing functions are used in the creation and verification of digital signatures. Why is this important in cybersecurity and secure communication?
- What is the purpose of salting passwords before hashing them?

Conclusion:

- Write what was performed in the experiment.
- Write the significance of the topic studied in the experiment.

References:

http://cse29-iiith.vlabs.ac.in/